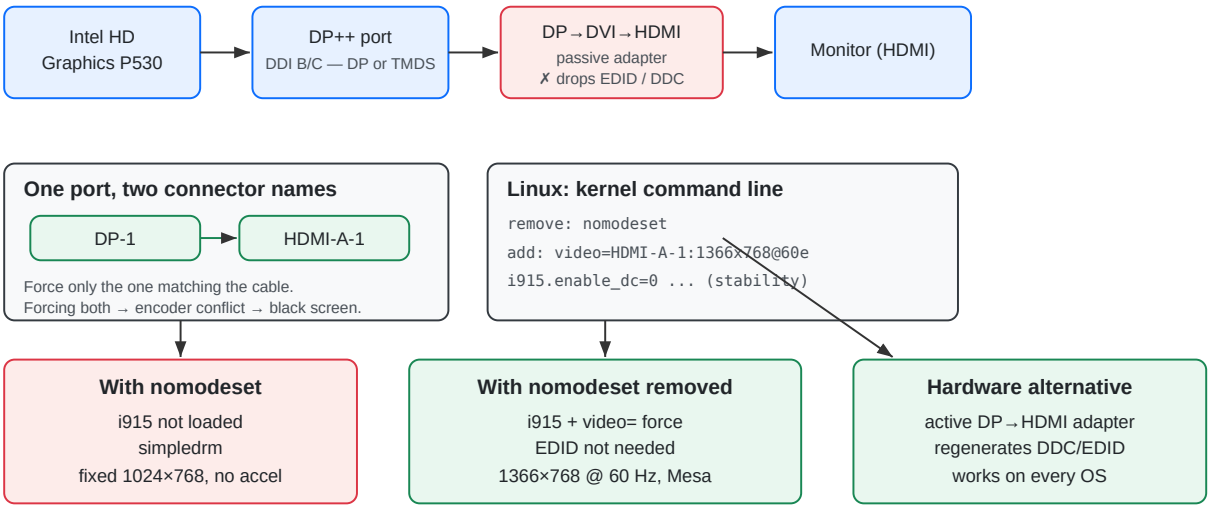
HP ProLiant ML10 Gen9

Graphics & Display Fix — Linux, Windows Server & more

Intel HD Graphics P530 • DP→DVI→HDMI • root cause, fix, scripts, verification

HP ProLiant ML10 Gen9 — display signal chain & failure points



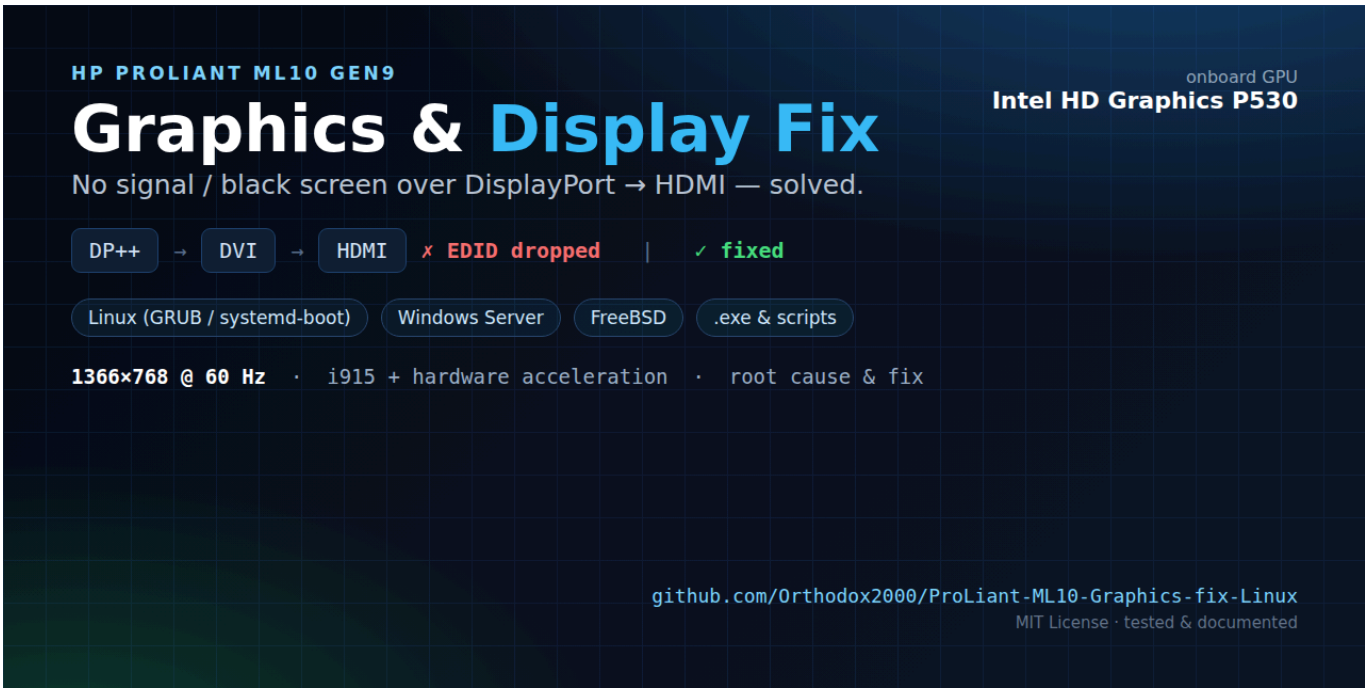
Root cause in one line

A passive DP→DVI→HDMI chain hides the monitor's EDID (GMBUS read fails).
Without EDID the driver never enables the output → "no signal"; nomodeset just hides it behind a 1024×768 framebuffer.
Fix: skip EDID entirely with video=<connector>:<mode>e, or use an active adapter that restores EDID.

Generated 2026-10-15
https://github.com/Orthodox2000/ProLiant-ML10-Graphics-fix-Linux
MIT License

Contents

1. HP ProLiant ML10 Gen9 — Graphics / Display Fix (Linux, Windows Server & more)
2. QUICKSTART
3. 1. Problem statement
4. 2. Diagnosis & evidence
5. 3. Root cause
6. 4. Linux solution (the fix)
7. 5. Distribution & bootloader guide
8. 6. Windows Server & Windows (10/11)
9. 7. Other operating systems
10. 8. Hardware, cables, adapters & alternative solutions
11. 9. Troubleshooting
12. 10. Verification
13. 11. Reference
14. 12. FAQ
15. Changelog



HP ProLiant ML10 Gen9 — Graphics / Display Fix (Linux, Windows Server & more)

> **Symptom:** the onboard Intel graphics (HD Graphics **P530**, `8086:191d`) works fine under `nomodeset` only at a fixed **1024×768**, but as soon as you remove `nomodeset` — or on Windows Server — the display goes **black / "no signal"**. This repository is the complete, tested, documented fix: root-cause analysis, ready-to-run scripts for many distributions, Windows Server instructions, hardware/cable alternatives, and full troubleshooting.

Tested reference machine: **HPE ProLiant ML10 Gen9**, Xeon E3-1225 v5, 2× DisplayPort 1.2, monitor over **DP → DVI → HDMI**, Linux Mint 22.x. Result: **1366×768 @ 60 Hz** with full hardware acceleration.

> 📄 **New here?** Read **QUICKSTART.md** first. 🖨️ **Printable / offline: docs/dist/ProLiant-ML10-Graphics-fix.pdf** (all documentation in one 49-page PDF) — see Screenshots.

TL;DR — the fix

The machine needs `i915` (the Intel KMS driver) to own the display, and the output must be **force-enabled** because the DP→DVI→HDMI adapter chain does not pass EDID reliably:

```
# Linux kernel command line (remove nomodeset, add):
i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 \
i915.enable_dp_mst=0 i915.disable_power_well=0 \
video=HDMI-A-1:1366x768@60e
```

Run the toolkit and let it do it for you:

```
./scripts/linux/i915-display-fix.sh status      # inspect
./scripts/linux/i915-display-fix.sh detect      # find your connector + mode
sudo ./scripts/linux/i915-display-fix.sh apply  # apply permanently
sudo reboot
```

On **Windows Server / Windows** there is no kernel command line — you install the Intel graphics driver and (if needed) use an *active* DisplayPort adapter. See **docs/06-windows-server.md**.

Table of contents

#	Document	What it covers
1	Problem statement	Symptoms, affected hardware, scope
2	Diagnosis & evidence	How it was found, commands, real logs
3	Root cause	`nomodeset`, EDID/GMBUS, DDI B dual-name, Skylake power wells
4	Linux solution	The fix, step-by-step, every parameter explained
5	Distro & bootloader guide	Ubuntu, Debian, Mint, Fedora, RHEL, Arch, openSUSE, NixOS, systemd-boot
6	Windows Server / Windows	Drivers, resolution, active adapter, RDP, PowerShell, .exe
7	Other OSes	FreeBSD, Proxmox/ESXi, TrueNAS, pfSense/OPNsense, illumos
8	Hardware, cables & adapters	Active vs passive DP, adapters, KVMs, alternative outputs
9	Troubleshooting	Decision tree for “still dark”, wrong resolution, etc.
10	Verification	Prove it worked, on any OS
11	Reference	Kernel params, `video=` syntax, `drm.debug`, glossary
12	FAQ	Sharp questions and answers

Repository layout

```
ProLiant-ML10-Graphics-fix-Linux/
├── README.md                        ← you are here
├── QUICKSTART.md                    ← 60-second version
├── LICENSE                          ← MIT
├── CITATION.cff                     ← "Cite this repository" metadata
├── AUTHORS                          ← maintainer + how to cite
├── CHANGELOG.md
├── Makefile                         ← build .exe, PDF, screenshots, lint
├── install.sh                       ← install Linux toolkit to /usr/local/sbin
├── docs/                            ← the full write-up (see table above)
│   └── dist/                        ← generated PDF + HTML (all docs combined)
├── scripts/
│   ├── linux/
│   │   ├── i915-display-fix.sh      ← portable toolkit (detect/apply/revert/discover)
│   │   ├── i915-collect.sh          ← gather diagnostics from a failing boot
│   │   ├── finalize-i915.sh         ← one-shot, non-interactive apply
│   │   └── README.md
│   ├── windows/
│   │   ├── Get-GraphicsInfo.ps1     ← inventory GPU + displays
│   │   ├── Set-DisplayResolution.ps1← set resolution / list modes
│   │   ├── Install-IntelGraphicsDriver.ps1
│   │   ├── Fix-Display.bat          ← double-click wrapper
│   │   ├── DisplayFix.exe           ← prebuilt native tool
│   │   ├── src/DisplayFix.c         ← source of the tiny native .exe
│   │   └── README.md
│   ├── freebsd/
│   │   ├── i915-freebsd.sh
│   │   └── README.md
│   └── doc/
│       ├── build-pdf.sh             ← Markdown -> HTML -> Chrome -> PDF
│       ├── make-screenshots.sh      ← render real output as terminal PNGs
│       └── make-banner.sh           ← render the social-preview banner
├── examples/                        ← real logs/cmdlines from the reference machine
├── assets/
│   ├── architecture.svg             ← signal-chain diagram
│   ├── social-preview.png           ← repo banner (1280x640)
│   ├── social/                      ← shareable post images (1200x627, 1080x1080)
│   └── screenshots/                 ← generated terminal captures
└── .github/workflows/ci.yml         ← shellcheck + syntax + Windows build CI
```

Why this repository exists

The ML10 Gen9 is a cheap, quiet Xeon tower with **two DisplayPort 1.2 outputs** on the motherboard and **no HDMI**. Most people connect a monitor meant for HDMI through a cheap **DP→DVI→HDMI** (or DP→HDMI) adapter. The result is a machine that "has no video" under a real graphics driver. Every forum answer says one of:

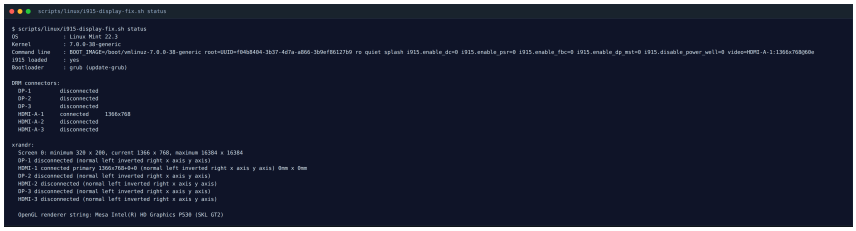
- "Use `nomodeset`" → you get 1024×768 forever and no GPU acceleration;
- "Use an active adapter" → correct, but nobody explains *why*, or how to make a passive one work;
- "The ML10 has no iGPU support" → **false**, the P530 is fully supported.

This repository documents the actual technical cause and gives a **software fix that works with a passive adapter**, plus the hardware fix if you prefer it, on **every OS we could test or verify**.

Screenshots

Everything below is the **real output of the reference machine**, captured with `scripts/doc/make-screenshots.sh` (it runs the commands and rasterises their output — nothing is mocked up).

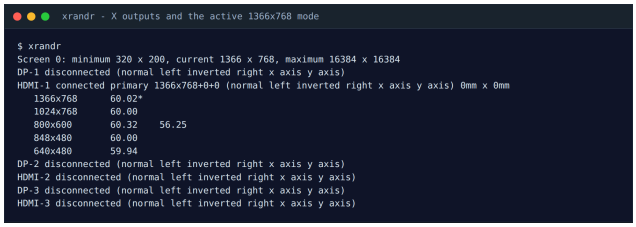
Toolkit status (`i915-display-fix.sh status`)



Connector detection (`… detect`)



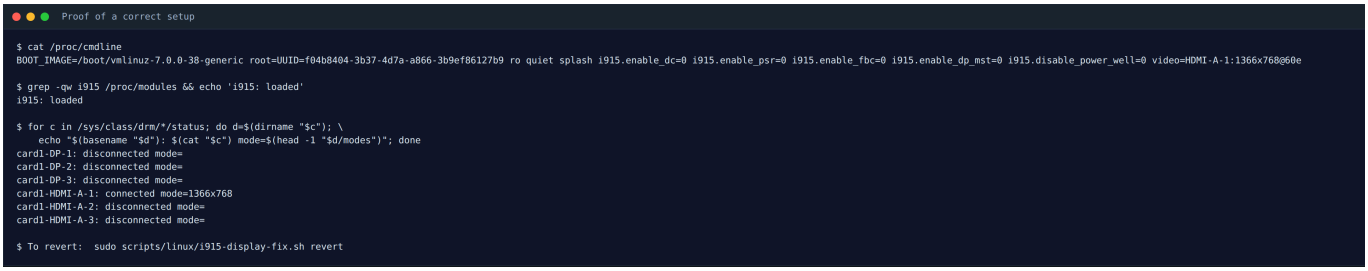
X outputs (`xrandr`)



Hardware acceleration (`glxinfo`)



Proof of a correct setup — `nomodeset` gone, `i915` loaded, one connector enabled at 1366×768:



Windows Server / Windows — the tools you run there (see `docs/06-windows-server.md`):

Windows Server / Windows - tools to run

```
C:\ProLiant-ML10-Graphics-fix> Fix-Display.bat info

== inventory ==
Get-CimInstance Win32_VideoController   ->  Name / DriverVersion / Resolution
Get-PnpDevice -Class Display            ->  is it "Microsoft Basic Display Adapter"?
WmiMonitorID (root\wmi)                 ->  did the OS read the monitor's EDID?

C:\ProLiant-ML10-Graphics-fix> Fix-Display.bat list
   1366x768 @ 60 Hz
   1280x720 @ 60 Hz
   1024x768 @ 60 Hz
    ...

C:\ProLiant-ML10-Graphics-fix> Fix-Display.bat set 1366 768
Setting \\.\DISPLAY1 to 1366x768 ...
Applied successfully.  (persisted to the registry)

C:\ProLiant-ML10-Graphics-fix> DisplayFix.exe --best
Best mode: 1366x768 @ 60 Hz
[+] \\.\DISPLAY1 set to 1366x768

# If only 640x480 / 1024x768 is available, the monitor's EDID is not reaching
# Windows -> use an ACTIVE DisplayPort -> HDMI adapter. See docs/06-windows-server.md
```

> The signal-chain diagram at the top of the PDF is `assets/architecture.svg`.

Affected / similar hardware

Anything with these Intel generations behaves the same way over a passive DP→DVI/HDMI chain:

Generation	iGPU examples	PCI IDs
Skylake (this repo)	HD Graphics **P530**, 530, Iris 540/550	`191d`, `1912`, `1916`, `1926/27`
Broadwell	HD 5500/5600, Iris Pro 6200	`1616`, `1622`, `1626`
Kaby Lake	HD 630, Iris Plus 640/650	`5912`, `5916`, `5926`
Coffee Lake	UHD 630	`3e92`, `3e9b`
Comet/Tiger Lake	UHD/Iris Xe	`9b41`, `9a49`

The same `nomodeset` + EDID-force-enable technique applies to all of them.

Safety

- Every script is **non-destructive by default** and keeps timestamped backups (`/var/backups/i915-display-fix/`).
- A guaranteed **recovery boot entry** (`nomodeset`) is always left in place, so a bad change is never permanent.

- `apply --dry-run` shows exactly what will change and touches nothing.

> ⚠️ The scripts edit the bootloader configuration. Read the relevant docs first, and keep a way to reach a console (physical keyboard or SSH enabled beforehand) if you are changing display settings on a machine you cannot see.

Citation

If this helped you (or you reference it), please cite it. The metadata lives in `CITATION.cff` (GitHub shows a **“Cite this repository”** button), and `AUTHORS` has a ready-made BibTeX entry.

```
@misc{orthodox2000_ml10gen9_graphics_fix,
  author       = {Orthodox2000},
  title        = {HP ProLiant ML10 Gen9 --- Graphics / Display Fix
                  (Linux, Windows Server \& more)},
  year         = {2026},
  version      = {1.1.0},
  howpublished = {GitHub repository},
  url          = {https://github.com/Orthodox2000/ProLiant-ML10-Graphics-fix-Linux},
  note         = {MIT License}
}
```

License

MIT — see LICENSE. Not affiliated with HPE or Intel. Use at your own risk; no warranty.

Contributing

Tested this on another board, distro or monitor? Please open a PR updating the matrices in `docs/05-distro-guide.md` and `docs/08-hardware-cables-adapters.md`.

QUICKSTART

The 60-second version. Full detail lives in `docs/` and `README.md`.

What this fixes

HP ProLiant **ML10 Gen9** (Intel **HD Graphics P530**) shows **no signal / black screen**, or is stuck at a fixed 1024×768, when the monitor is connected through the motherboard's **DisplayPort** via a **DP → DVI → HDMI** adapter.

Cause: the passive adapter hides the monitor's EDID, so the driver never enables the output. `nomodeset` "works" only by abandoning the GPU entirely.

Linux — 3 commands

```
cd scripts/linux
./i915-display-fix.sh detect                # tells you the connector + mode
sudo ./i915-display-fix.sh apply            # applies it permanently
sudo reboot
```

If the screen is **dark right now** and you don't know the connector name:

```
sudo ./i915-display-fix.sh discover         # adds a safe "no nomodeset" GRUB entry
sudo reboot                                 # pick "i915 DISCOVER" at the menu
./i915-display-fix.sh detect                # then apply with what it prints
```

Verify after rebooting:

```
./i915-display-fix.sh status
```

Revert any time:

```
sudo ./i915-display-fix.sh revert
```

The raw kernel parameters (any distro)

Remove `nomodeset`; add these to the kernel command line:

```
i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable_dp_mst=0 i915.disable_power_well=0 video=
```

- Replace **`HDMI-A-1`** with your connector (`DP-1` if the monitor is native DP).
- Replace **`1366x768`** with your monitor's native resolution.

- Keep the trailing `e` (force-enable even when "disconnected").
- **Never** force both `DP-1` and `HDMI-A-1` — they are the *same* physical port.

Per-distro regeneration: see `docs/05-distro-guide.md`.

Windows / Windows Server

There is no kernel command line — install the **Intel graphics driver** and, if the monitor still isn't detected, use an **active** DP→HDMI adapter.

```
Fix-Display.bat info          :: are you on the Basic Display Adapter?
Fix-Display.bat list          :: supported modes
Fix-Display.bat set 1366 768  :: set + persist a mode
Fix-Display.bat best          :: highest mode the display reports
```

Full guide: `docs/06-windows-server.md`.

Other OSes

OS	Quick fix
FreeBSD / pfSense / TrueNAS CORE	`i915kms_load="YES"` in `/boot/loader.conf` (script)
Proxmox / TrueNAS SCALE / Unraid	Linux fix above
VMware ESXi	manage over the network (no iGPU driver)

Details: `docs/07-other-os.md`.

Always works: the hardware fix

A **passive** DP→HDMI/DVI plug cannot pass EDID. An **active** one can, and then Linux, Windows and FreeBSD all just work — no software changes needed. See `docs/08-hardware-cables-adapters.md`.

Install the toolkit system-wide (optional)

```
sudo ./install.sh        # installs to /usr/local/sbin
sudo i915-display-fix status
```

Need help?

- Still dark → `docs/09-troubleshooting.md`
- Prove it worked → `docs/10-verification.md`

- Collect a report to share → `bash scripts/linux/i915-collect.sh /tmp/i915`

1. Problem statement

1.1 Summary

On an **HPE ProLiant ML10 Gen9** with the onboard **Intel HD Graphics P530** (`8086:191d`), a monitor connected through the motherboard's **DisplayPort** — in practice nearly always via a **DP → DVI → HDMI** or **DP → HDMI** adapter — does not produce a picture under a normal Linux graphics stack.

Two distinct behaviours are reported:

Configuration	Result
`nomodeset` on the kernel command line (the "safe" default)	Boots to a **fixed 1024×768** desktop. No GPU acceleration. Resolution cannot be changed in software.
`nomodeset` removed ("normal" boot)	**Black screen / "no signal"** from the motherboard. Sometimes a dark mouse cursor flickers in a corner.
Windows Server / Windows	Often **no display at all** after driver install, or a "generic" 640×480 desktop.

1.2 Why people hit this

The ML10 Gen9 motherboard exposes **two DisplayPort 1.2** connectors and **no HDMI/DVI/VGA**. The overwhelming majority of monitors in this price bracket are HDMI. So the realistic chain is:

```
[motherboard DP] ──► [DP→DVI plug] ──► [DVI→HDMI cable] ──► [monitor HDMI]
```

At least three things are wrong with that chain on this hardware:

1. **`nomodeset` is not a resolution fix** — it disables kernel mode-setting entirely, leaving you with the firmware's fixed framebuffer.
2. **EDID (the monitor's "capabilities chip") is not passed reliably** through cheap passive adapters, so the driver cannot see that a monitor is attached.
3. On Intel, a **dual-mode ("DP++") port appears twice**: once under its DP name (`DP-1`) and once under its HDMI name (`HDMI-A-1`). Force-enabling the wrong one, or both, makes the driver fight itself.

1.3 Scope of this repository

In scope:

- Every Linux distribution we can cover with a bootloader-based fix (GRUB and systemd-boot), including per-distro commands.
- Windows Server 2012 R2 → 2025 and Windows 10/11.
- FreeBSD and the common virtualisation/"NAS" platforms.

- Hardware alternatives (active adapters, KVMs, alternative connectors).

Out of scope (documented only):

- Reflashing the BIOS/firmware.
- Adding a discrete GPU.
- Multi-monitor tiling edge cases.

1.4 Reference hardware (used for every measurement here)

Item	Value
System	HPE ProLiant ML10 Gen9 (`ML10Gen9`), BIOS 1.010
CPU	Intel Xeon E3-1225 v5 @ 3.30 GHz (4 cores, VT-x)
iGPU	Intel HD Graphics **P530**, PCI `8086:191d`, subsystem `8086:2212`
Display outputs	2 × DisplayPort 1.2 (DP++ capable, per VBT)
Display	1366×768 monitor on HDMI
Cable chain	DP → DVI plug → DVI→HDMI cable → monitor
OS (reference)	Linux Mint 22.3, kernel 7.0.0-38-generic, Mesa 25.x
Other kernels tested	6.14-generic (RHEL/Debian-style naming irrelevant)

Raw captures live in `../examples/`.

2. Diagnosis & evidence

This is the actual investigation that produced the fix, in order, with the real commands and their outputs. You can reproduce every step on your own ML10.

2.1 Identify the GPU and driver

```
$ lspci -nnk | grep -A3 -iE 'VGA|Display'
00:02.0 VGA compatible controller [0300]: Intel Corporation HD Graphics P530 [8086:191d] (rev 06)
        DeviceName:  Onboard IGD
        Subsystem: Intel Corporation HD Graphics P530 [8086:2212]
        Kernel driver in use: i915
```

So the hardware *is* supported by `i915`. (`00:02.0` is the standard location of the Intel iGPU.)

2.2 Confirm what is *actually* driving the display

```
$ cat /proc/cmdline
... ro quiet splash nomodeset
$ grep -w i915 /proc/modules || echo "i915 NOT loaded"
i915 NOT loaded
$ lsmod | grep -E 'i915|simpledrm'
simpledrm    ...
```

With `nomodeset`, only the **firmware framebuffer** (`simpledrm`) is active. That is why the resolution is fixed and unchangeable: nothing is programming the CRTC.

2.3 Prove that `nomodeset` is the blocker

```
$ sudo modprobe i915
modprobe: ERROR: could not insert 'i915': No such device
```

`i915` refuses to bind **while `nomodeset` is active** — it is not a missing driver, it is a kernel policy flag (`video_firmware_drivers_only()`), and it is **boot-time only**. No amount of `rmmod`/`modprobe` will fix it; you must change the kernel command line and reboot.

2.4 Look at the connector the firmware left enabled

Booting once **without** `nomodeset` and with verbose DRM logging (`drm.debug=0x1e`) makes `i915` speak. Two things jump out.

(a) The port really is dual-mode

```
i915 ... [drm:intel_bios_init] Port B VBT info: CRT:0 DVI:1 HDMI:1 DP:1 DP++:1
i915 ... [drm:intel_bios_init] Port C VBT info: CRT:0 DVI:1 HDMI:1 DP:1 DP++:1
i915 ... [drm:intel_bios_init] Port D VBT info: CRT:0 DVI:1 HDMI:1 DP:1 DP++:1
```

`DP++:1` on ports B/C/D means the GPU **can** emit TMDS (DVI/HDMI) through a passive adapter. The adapter is not the (only) problem.

(b) EDID does not come through cleanly

```
i915 ... [drm:intel_hdmi_set_edid] HDMI GMBUS EDID read failed, retry using GPIO bit-banging
i915 ... [drm:intel_hdmi_set_edid] HDMI GMBUS EDID read failed, retry using GPIO bit-banging
...
```

Hundreds of these. The GMBUS (hardware DDC) read fails; the kernel falls back to **bit-banging** the DDC lines, which is slow and racy at boot. If it loses the race, the connector is treated as "disconnected" and the output stays dark.

(c) A dual-mode port has TWO names

```
i915 ... [drm:intel_dp_init_connector]   Adding DP connector on [ENCODER:109:DDI B/PHY B]
i915 ... [drm:intel_hdmi_init_connector] Adding HDMI connector on [ENCODER:109:DDI B/PHY B]
...
i915 ... [CONNECTOR:120:HDMI-A-1] hw state readout: enabled
```

`DP-1` and `HDMI-A-1` are the **same physical port** (DDI B). The BIOS left `HDMI-A-1` enabled, i.e. that is the connector your cable is on.

2.5 The first, wrong "fix" — and why it failed

Forcing **all** candidate connectors:

```
video=DP-1:1366x768@60e video=DP-2:1366x768@60e \
video=HDMI-A-1:1366x768@60e video=HDMI-A-2:1366x768@60e
```

produced **"signal, but a black screen with a flickering cursor"**. The kernel explains why:

```
i915 ... [drm:handle_conflicting_encoders] [ENCODER:109:DDI B/PHY B] in use on
        [CRTC:58:pipe A], disabling [CONNECTOR:120:HDMI-A-1]
i915 ... [drm:update_connector_routing] Disabling [CONNECTOR:110:DP-1]
```

`DP-1` grabbed the DDI B encoder, so the driver **kept disabling the real monitor** (`HDMI-A-1`). Xorg then saw four outputs (two of them phantom) and set them all to 640×480 — hence black screen, occasional cursor.

2.6 The correct fix

Force **only the connector that matches the cable**:

```
video=HDMI-A-1:1366x768@60e
```

Result on the reference machine:

```
$ cat /proc/cmdline
... quiet splash i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 \
    i915.enable_dp_mst=0 i915.disable_power_well=0 video=HDMI-A-1:1366x768@60e

$ head -1 /sys/class/drm/card1-HDMI-A-1/modes
1366x768
$ xrandr | grep connected
HDMI-1 connected primary 1366x768+0+0 ...
$ glxinfo | grep 'OpenGL renderer'
OpenGL renderer string: Mesa Intel(R) HD Graphics P530 (SKL GT2)
```

2.7 Evidence files in this repo

File	Contents
`examples/lspci-gpu.txt`	GPU + driver
`examples/cpu.txt`	CPU flags
`examples/system.txt`	Board + BIOS
`examples/working-cmdline.txt`	The working kernel line
`examples/drm-connectors-working.txt`	Connector states
`examples/xrandr-working.txt`	X outputs + modes
`examples/i915-vbt-ports.txt`	DP++ proof
`examples/i915-edid-noise.txt`	EDID/GMBUS failure
`examples/i915-encoder-conflict.txt`	The dual-name trap
`examples/modeline-1366x768.txt`	Mode timings
`examples/modinfo-i915.txt`	Module info

To gather the same data on your machine, run `scripts/linux/i915-collect.sh`.

3. Root cause

Four independent facts combine into the failure. Understanding them tells you which fix will work and which will not.

3.1 `nomodeset` is a global, boot-time kill switch

`nomodeset` sets the kernel variable `video_firmware_drivers_only()`. While it is set, **DRM/KMS drivers are not allowed to take over** the display; only the firmware's simple framebuffer (`simpledrm`, driven by the BIOS GOP) is used.

Consequences:

- Resolution is whatever the firmware set at boot — here **1024×768** — and cannot be changed.
- No hardware acceleration; the desktop runs on software rendering (`llvmpipe`).
- `modprobe i915` fails with `No such device` (the driver declines to probe).
- **It cannot be undone at runtime.** It is read once at boot.

```
flowchart LR
  A[kernel cmdline] -->|has nomodeset| B[video_firmware_drivers_only = true]
  B --> C[i915 refuses to bind]
  C --> D[simpledrm / fixed 1024x768]
  A -->|no nomodeset| E[i915 binds]
  E --> F[KMS: real modes, acceleration]
```

3.2 EDID does not survive the DP→DVI→HDMI chain

EDID is the small data block a monitor exposes over the DDC channel (I²C). The driver needs it to know the monitor exists and what modes it supports.

On Intel, DDC is normally read through **GMBUS** (a hardware I²C controller). Cheap passive adapters — and even some active ones — break or degrade DDC, so:

```
intel_hdmi_set_edid: HDMI GMBUS EDID read failed, retry using GPIO bit-banging
```

appears hundreds of times. The fallback (bit-banging GPIO) is slow and, during early boot, often loses the race with the desktop starting. When that happens the connector is reported **disconnected** → the output is never turned on → **"no signal"**.

> This is why the *hardware* fix is "use an active adapter" (it regenerates DDC), and why the *software* fix is "force the connector on and tell the driver the mode" so it does not need EDID at all.

3.3 A dual-mode (DP++) port has two names

Intel DP++ ports can carry native DisplayPort **or** TMDS (DVI/HDMI) depending on what is plugged in. Because of that, the driver registers **two connectors on one physical port**, e.g.:

```
port DDI B ─┬─ connector DP-1        (native DP)
            └─ connector HDMI-A-1    (DP++ / TMDS)
```

The BIOS leaves the one matching the actual cable enabled. On the reference machine `hd state readout: enabled` was on **`HDMI-A-1`**, so the cable is on the DP++ path in HDMI mode.

Trap: force-enabling *both* names makes two connectors share one encoder:

```
handle_conflicting_encoders: [DDI B/PHY B] in use on pipe A, disabling HDMI-A-1
```

…and the driver disables the real monitor in a loop. This produces the notorious “signal present, black screen, flashing cursor”.

3.4 Skylake-era power management can blank an output

Skylake’s display engine has aggressive power features that are known to cause black screens / flicker on external adapters:

- **DC states** (display C-states) — `i915.enable_dc`
- **PSR** (Panel Self Refresh) — `i915.enable_psr`
- **FBC** (Frame Buffer Compression) — `i915.enable_fbc`
- **Power wells** — `i915.disable_power_well`

Disabling them (as this repo does) trades a little idle power for stability. On a desktop/server that is the right trade.

3.5 Minimal working condition

Putting it together, the display lights up iff **all** of these are true:

1. `nomodeset` is **absent** (so `i915` binds).
2. The correct **connector name** is force-enabled (`video=<name>:<mode>e`), compensating for the missing EDID.
3. A **supported mode** is requested (here 1366×768@60).
4. Optionally, Skylake power features are disabled for stability.

Note that (2) is only necessary because of (3.2). With a good **active adapter**, plain boot without `video=` usually works, because EDID is readable.

4. Linux solution (the fix)

This document is the canonical, step-by-step procedure. It assumes GRUB (the default on Ubuntu/Mint/Debian/Fedora/Arch/openSUSE); systemd-boot and distro-specific commands are in 05-distro-guide.md.

> **Two ways to run it:** the automated toolkit (`scripts/linux/i915-display-fix.sh`), or the manual steps below. Both do exactly the same thing.

4.1 Automated (recommended)

```
cd scripts/linux
chmod +x i915-display-fix.sh

# 1. Inspect the current state (read-only, safe):
./i915-display-fix.sh status

# 2. If i915 is running, it can tell you the connector + mode:
./i915-display-fix.sh detect
#    -> "Apply it with: sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768"

# 3. Preview, then apply:
sudo ./i915-display-fix.sh apply --dry-run
sudo ./i915-display-fix.sh apply

# 4. Reboot (do NOT touch the menu; it auto-boots the fixed entry):
sudo reboot
```

If the machine is currently dark and you do not know the connector name, use the two-phase workflow in 09-troubleshooting.md.

4.2 Manual (GRUB) — the exact steps

Step 1 — Find the connector name

If you can already boot to a desktop with `nomodeset`, you still cannot see the real connector name (only `Unknown-1` from `simpledrm`). The reliable way is to boot once **without** `nomodeset` and read the state, even if the screen is dark:

```
# after booting without nomodeset, over SSH or a second console:
for c in /sys/class/drm/*/status; do
  echo "$(basename "$(dirname "$c")"): $(cat "$c")"
done
```

You want the connector whose status is `connected` — typically `HDMI-A-1` or `DP-1`. (If nothing is `connected`, see the discover workflow.)

Step 2 — Edit `/etc/default/grub`

```
sudo cp -a /etc/default/grub /etc/default/grub.bak-$(date +%Y%m%d-%H%M%S)
sudo nano /etc/default/grub
```

Change the line:

```
-GRUB_CMDLINE_LINUX_DEFAULT="quiet splash nomodeset"
+GRUB_CMDLINE_LINUX_DEFAULT="quiet splash i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable
```

Notes:

- Delete `nomodeset`.
- `HDMI-A-1` → your connector name from Step 1.
- `1366x768` → your monitor's native resolution (see 10-verification.md).
- Keep the `e` suffix (force-enable even if "disconnected").

Optional, to make the menu visible long enough to reach a recovery entry:

```
+GRUB_TIMEOUT_STYLE=menu
+GRUB_TIMEOUT=10
```

Step 3 — Regenerate the GRUB config

Debian/Ubuntu/Mint:

```
sudo update-grub
```

Fedora/RHEL/openSUSE:

```
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

Arch:

```
sudo grub-mkconfig -o /boot/grub/grub.cfg
```

Step 4 — Add a recovery entry (strongly recommended)

Create `/etc/grub.d/42_i915_fix_recovery` (mode `0755`) so you always have a one-keystroke way back to a working 1024×768 desktop:

```
KREL=$(uname -r)
UUID=$(findmnt -no UUID /)
sudo tee /etc/grub.d/42_i915_fix_recovery >/dev/null <<EOF
#!/bin/sh
exec tail -n +3 \$0
menuentry "Recovery - safe graphics (nomodeset)" --class gnu-linux --class os {
    recordfail
    load_video
    set gfxpayload=keep
    insmod gzio
    insmod part_gpt
    insmod ext2
    search --no-floppy --fs-uuid --set=root $UUID
    linux  /boot/vmlinuz-$KREL root=UUID=$UUID ro quiet splash nomodeset
    initrd /boot/initrd.img-$KREL
}
EOF
sudo chmod 755 /etc/grub.d/42_i915_fix_recovery
sudo update-grub
```

> ⚠️ **Never** leave `*.bak` copies inside `/etc/grub.d/`. GRUB executes every executable file in that directory, and a backup containing a `menuentry` line becomes a **duplicate menu entry**. Keep backups in `/root/` or `/var/backups/`.

Step 5 — Reboot and verify

```
sudo reboot
# after it comes back:
cat /proc/cmdline
xrandr | grep connected         # expect "<name> connected primary 1366x768+0+0"
```

4.3 What each parameter does

Parameter	Effect	Why it is here
(remove) `nomodeset`	Lets `i915` bind	**The** essential change
`video=HDMI-A-1:1366x768@60e`	Force-enable connector + mode	Works around missing EDID
`i915.enable_dc=0`	Disable display C-states	Skylake analog power glitches
`i915.enable_psr=0`	Disable Panel Self Refresh	No panel here; avoids blanking
`i915.enable_fbc=0`	Disable frame-buffer compression	Avoids flicker on adapters
`i915.enable_dp_mst=0`	Disable DP Multi-Stream	No MST hub; simplifies link setup
`i915.disable_power_well=0`	Keep power wells always on	Maximum stability

Only `nomodeset`-removal and the `video=` token are strictly required in most cases. The `i915.*` flags are belt-and-braces stability; drop them if you prefer default power behaviour and it still works.

4.4 Reverting

```
sudo cp -a /etc/default/grub.bak-<timestamp> /etc/default/grub
sudo rm -f /etc/grub.d/42_i915_fix_recovery
sudo update-grub
sudo reboot
```

Or with the toolkit:

```
sudo ./i915-display-fix.sh revert
```

4.5 Changing the resolution later

Temporarily (current session):

```
xrandr --output HDMI-1 --mode 1366x768
```

Permanently: change `1366x768` in the `video=` token and re-run Step 3, or add an Xorg configuration / a desktop startup command (see 05-distro-guide.md).

5. Distribution & bootloader guide

The toolkit edits the **bootloader**, so one script covers every GRUB-based and systemd-boot-based distribution. `status` and `detect` are read-only and safe everywhere; `apply` / `revert` / `discover` need root.

OS / family	Versions covered	Bootloader	Regeneration command
Ubuntu / Linux Mint / Pop!_OS / elementary / Zorin	20.04, 22.04, 24.04, LM 20–22	GRUB	`update-grub`
Debian	10, 11, 12, 13	GRUB	`update-grub`
Fedora / RHEL / CentOS Stream / Rocky / Alma	8, 9, 10	GRUB2	`grub2-mkconfig -o /boot/grub2/grub.cfg`
Arch / Manjaro / EndeavourOS	rolling	GRUB **or** systemd-boot	`grub-mkconfig -o /boot/grub/grub.cfg`
openSUSE Leap / Tumbleweed / SLES	15.x	GRUB2	`grub2-mkconfig -o /boot/grub2/grub.cfg`
NixOS	any	GRUB/systemd-boot	declarative (`configuration.nix`)
Void / Gentoo / Alpine	any	various	manual

The script auto-detects the distribution (`/etc/os-release`), the correct regeneration tool, and the GRUB config path. **Kernel version does not matter** — only `uname -r` is used to locate the image for the optional recovery entry.

5.1 Ubuntu / Linux Mint / Debian / Pop!_OS

```
./i915-display-fix.sh detect
sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
sudo reboot
```

Equivalent manual edit of `/etc/default/grub` :

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable_
```

then `sudo update-grub` .

5.2 Fedora / RHEL / Rocky / AlmaLinux / CentOS Stream

```
sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
# or, the distro-native way:
sudo grubby --update-kernel=ALL --remove-args=nomodeset \
  --args='i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable_dp_mst=0 i915.disable_power_we
```

- **UEFI and BIOS both work.** The GRUB config on UEFI Fedora is still `/boot/grub2/grub.cfg` (the `EFI/fedora/grub.cfg` file is a stub that loads it).
- `grubby` is present on RHEL-family systems and is the safest option; it edits all kernel entries at once.
- After a **kernel update**, `grubby --update-kernel=ALL` persists, but a plain `grub2-mkconfig` edit of `/etc/default/grub` also persists. Both are fine.

5.3 Arch / Manjaro / EndeavourOS

Detect which bootloader you use:

```
[ -d /boot/loader/entries ] && echo systemd-boot || echo grub
```

GRUB:

```
sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
```

systemd-boot (the toolkit edits `/boot/loader/entries/*.conf`):

```
sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
```

Manual equivalent — edit the `options` line of your entry, e.g. `/boot/loader/entries/arch.conf`:

```
options root=UUID=... rw quiet i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable_dp_mst=0 i
```

Add a recovery entry by copying the file and appending `nomodeset`:

```
sudo cp /boot/loader/entries/arch.conf /boot/loader/entries/arch-recovery.conf
sudo sed -i 's/$/ nomodeset/' /boot/loader/entries/arch-recovery.conf
```

5.4 openSUSE Leap / Tumbleweed / SLES

```
sudo ./i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
# manual:
sudo sed -i 's/^GRUB_CMDLINE_LINUX_DEFAULT=.*/GRUB_CMDLINE_LINUX_DEFAULT="... video=HDMI-A-1:1366x768@60e"
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
```

YaST → *Boot Loader* → *Kernel Parameters* also works if you prefer a GUI.

5.5 NixOS (declarative — the script cannot edit it safely)

In `/etc/nixos/configuration.nix`:

```
{
  boot.kernelParams = [
    "i915.enable_dc=0"
    "i915.enable_psr=0"
    "i915.enable_fbc=0"
    "i915.enable_dp_mst=0"
    "i915.disable_power_well=0"
    "video=HDMI-A-1:1366x768@60e"
  ];
  # keep several generations so you can roll back at the boot menu
  boot.loader.grub.configurationLimit = 10;
  # never put "nomodeset" here:
  # boot.kernelParams = [ "nomodeset" ];   # <-- do NOT do this
}
```

Then `sudo nixos-rebuild switch`. Roll back with `nixos-rebuild switch --rollback` or by picking an older generation in the boot menu.

5.6 Void / Gentoo / Alpine and other bootloaders

`apply` prints the parameters to use and leaves the bootloader alone when it does not recognise it. Add these to your kernel command line:

```
i915.enable_dc=0 i915.enable_psr=0 i915.enable_fbc=0 i915.enable_dp_mst=0 i915.disable_power_well=0 video=
```

- **Void:** `/etc/default/grub` + `sudo update-grub` (same as Debian), or edit `/boot/loader/entries/` for systemd-boot.
- **Gentoo:** often a hand-written `/boot/grub/grub.cfg`; append the parameters to the `linux` line.
- **Alpine:** `/etc/update-extlinux.conf` (extlinux) or the GRUB config; append to `default_kernel_opts` / the kernel line.
- **rEFInd:** edit `/boot/refind_linux.conf`.

Persisting the resolution in X

The `video=` force sets the mode at boot, which is usually enough for GDM/SDDM/LightDM. If your desktop still starts at a lower mode, pin it per-user or system-wide.

Per-user (all desktops): add a desktop autostart script:

```
mkdir -p ~/.config/autostart
cat > ~/.config/autostart/i915-resolution.desktop <<'EOF'
[Desktop Entry]
Type=Application
Name=Set 1366x768
Exec=sh -c "sleep 3; xrandr --output HDMI-1 --mode 1366x768 || xrandr --output HDMI-1 --auto"
X-GNOME-Autostart-enabled=true
EOF
```

System-wide (Xorg, all users): `/etc/X11/xorg.conf.d/20-i915-mode.conf`:

```
Section "Monitor"
    Identifier "HDMI-1"
    Option "PreferredMode" "1366x768"
EndSection
```

(Requires the `modesetting` driver, which is the default for Intel.)

Wayland (GNOME/KDE): the kernel `video=` mode is honoured at boot; change the mode from the desktop’s Display settings.

6. Windows Server & Windows (10/11)

Windows does not use a kernel command line, so the Linux fix does not apply. The same machine, cable chain and failure show up differently here, and the solution is a combination of **driver** and **hardware** work.

6.1 Why Windows Server shows no/ugly video on this box

1. **Windows Server ships a generic driver.** Until a real graphics driver is installed, Windows uses the **Microsoft Basic Display Adapter** (`msbasicdisplay`). That adapter: - ignores most of the GPU, - caps the mode list to what the firmware/EDID provides, - with a **passive DP→DVI→HDMI chain that hides EDID**, often leaves you at **640×480 or 1024×768**.
2. **Intel does not officially support consumer iGPUs on Windows Server.** There is no SKU of the HD Graphics P530 driver labelled “Windows Server 2022”. The **Windows 10/11 64-bit** driver can usually still be installed manually (“Have Disk”), but you must bypass the compatibility check.
3. **The same EDID problem as Linux.** The DP++ port is fine, but a passive adapter can drop the DDC channel, so Windows cannot read the monitor. This is the root of “640×480 only” and “no signal”.

> Practical consequence: on Windows the **single most effective fix is an *active* DisplayPort adapter** (see 08-hardware-cables-adapters.md). Driver installation gets you acceleration and correct modes; the adapter fixes detection.

6.2 Solution A — install the Intel graphics driver (recommended first step)

What you need

Item	Value
GPU	Intel HD Graphics **P530**, `VEN_8086&DEV_191D`
Last Windows 10/11 x64 driver for Skylake (Gen9)	Intel **15.45.x** series, e.g. `15.45.34.5174`, INF name `igdlh64.inf`
Alternative	Intel **DCH** driver `31.0.101.x` (Skylake support is limited/vendor-specific)
HP option	Search HP support for “ML10 Gen9 Intel Graphics” SoftPaq (may cover Win Server 2012 R2/2016)

This is a **standard INF/`pnputil` driver install** — the same mechanism as any other Windows disk or device driver — so it works on Windows 10/11 and Windows Server 2012 R2 → 2025, GUI *or* command line, attended *or* unattended.

Method 1 — Device Manager "Have Disk" (works on Server 2016/2019/2022)

1. Download and **extract** the Intel driver package (`.exe` → run it and let it extract to `C:\SWSetup\...`, or use 7-Zip; the `.inf` files are in a subfolder such as `Graphics`).
2. `Device Manager` → **Display adapters** → *Microsoft Basic Display Adapter* → **Update driver** → **Browse my computer** → **Let me pick from a list** → **Have Disk…**
3. Point to `igdlh64.inf` in the extracted folder.
4. Choose **Intel(R) HD Graphics P530** (or 530) and accept the compatibility warning.
5. Reboot. Verify:

```
Get-CimInstance Win32_VideoController |
  Select-Object Name, DriverVersion, CurrentHorizontalResolution, CurrentVerticalResolution
```

You want `Intel(R) HD Graphics ...` and your native resolution, not `Microsoft Basic Display Adapter`.

> If the INF refuses to install on Server 2022/2025 because of the OS version, either (a) use the Windows 10/11 driver of the *same WDDM generation*, or (b) temporarily enable **test signing** / driver signature enforcement bypass, install, then re-enable. Prefer (a).

Method 2 — `pnputil` (scriptable, no GUI)

```
# Stage and install the INF (admin PowerShell):
pnputil /add-driver "C:\Drivers\Intel\Graphics\igdlh64.inf" /install
# If it installs as a package but does not bind:
pnputil /enum-drivers | Select-String -Context 0,4 igdlh
# then assign it in Device Manager, or:
Get-PnpDevice -Class Display | Enable-PnpDevice -Confirm:$false
```

`scripts/windows/Install-IntelGraphicsDriver.ps1` automates the staging and points out the manual Device Manager step if binding is refused. Two one-click/scriptable entry points ship with it:

```
:: double-click wrapper: auto-elevates and auto-detects the extracted driver
scripts\windows\Install-IntelGraphicsDriver.bat

:: or via the general wrapper (install / drivers subcommands)
Fix-Display.bat install
Fix-Display.bat install C:\SWSetup\Intel\Graphics
Fix-Display.bat drivers

:: scriptable, no GUI — suitable for unattended installs / Server Core
powershell -ExecutionPolicy Bypass -File Install-IntelGraphicsDriver.ps1 -Path C:\SWSetup\Intel\Graphics
```

If you omit `-Path`, the script searches the usual extraction locations (`%SystemDrive%\SWSetup\Intel*`, `%SystemDrive%\Drivers\Intel*`, `Downloads`, the current folder). Like any `pnputil` driver, this can be driven from `unattend.xml`, an imaging task sequence, or a login script.

Method 3 — HP SoftPaq

HP occasionally publishes a Windows Server graphics driver for the ML10 Gen9 (usually for Server 2012 R2/2016). Check HP support by serial number; if it exists it installs cleanly with no compatibility hacks.

6.3 Solution B — fix detection (the EDID problem)

Even with the right driver, a passive adapter can leave Windows unable to read the monitor. In order of preference:

1. **Use an active DP adapter** (see §6.4). This is the real fix and also helps Linux.
2. **Force a mode in the registry.** Windows stores display configuration under:

`HKLM\SYSTEM\CurrentControlSet\Control\GraphicsDrivers\Configuration`

Each subkey holds a `PrimSurfSize.cx` / `PrimSurfSize.cy` and per-monitor `ActiveSize.cx` / `.cy`. Setting them to `1366` / `768` and rebooting will make Windows *try* that mode (it only sticks if the adapter/EDID allows it). Back up the key first. This is a workaround, not a guarantee. 3. **Set the mode at logon** with a script (see §6.5). Works when the mode exists in the list but Windows picks a lower one.

6.4 Solution C — hardware: the active adapter

HP documents that the ML10 Gen9 DisplayPorts expect an **active** DP adapter. Active adapters contain a level-shifter/converter chip (DisplayPort → TMDS) and regenerate DDC, so both Windows and Linux see a proper monitor.

Buy one of:

- **Active DP → HDMI** (the cleanest: one cable), or
- **Active DP → DVI**, then a DVI→HDMI cable.

Look for the words **“active”** / “DP++ to HDMI active” in the product listing. A passive “DP to HDMI” plug has no chip and will reproduce this exact bug.

See 08-hardware-cables-adapters.md for details and alternatives.

6.5 Registry / PowerShell resolution control

Built-in (some Server versions)

```
Get-DisplayResolution           # if the cmdlet exists
Set-DisplayResolution -Width 1366 -Height 768
```

(These cmdlets exist on Server Core in several Server releases; if `Get-Command` cannot find them, use the next method.)

Portable (works on every Windows / Server)

Use the provided scripts:

```
# inventory: GPU, driver, monitors, and every mode each output supports
powershell -ExecutionPolicy Bypass -File scripts\windows\Get-GraphicsInfo.ps1

# list modes, then set one (uses ChangeDisplaySettingsEx via P/Invoke)
powershell -ExecutionPolicy Bypass -File scripts\windows\Set-DisplayResolution.ps1 -List
powershell -ExecutionPolicy Bypass -File scripts\windows\Set-DisplayResolution.ps1 -Width 1366 -Height 768
```

Or the tiny native tool (see §6.6):

```
DisplayFix.exe --list
DisplayFix.exe --set 1366 768
DisplayFix.exe --best
```

Third-party CLIs (if you prefer)

- **QRes** (Anders Kjersem): `QRes.exe /x:1366 /y:768`
- **NirCmd** (NirSoft): `nircmd.exe setdisplay 1366 768 32`
- **DisplayChanger** (12noon): `dc64.exe -width=1366 -height=768`

These are convenient for logon scripts / Task Scheduler “At log on”.

6.6 The `DisplayFix.exe` native tool

Because a native binary is handy for logon scripts, unattended setups and Server Core, this repo includes its **source** and a build recipe:

```
scripts/windows/src/DisplayFix.c      # ~200 lines, Win32 only, no dependencies
```

Features: - `--list` — enumerate adapters and every supported mode. - `--set W H [Hz] [device]` — apply a mode and persist it (`CDS_UPDATEREGISTRY`). - `--best [Hz] [device]` — pick the highest mode the adapter reports. - `--current` — print the active mode.

Build it (on any Linux with mingw-w64)

```
sudo apt-get install -y mingw-w64        # Debian/Ubuntu
make windows                              # produces scripts/windows/DisplayFix.exe
```

or directly:

```
x86_64-w64-mingw32-gcc -O2 -municode -o DisplayFix.exe src/DisplayFix.c -lgdi32 -luser32
```

Build it on Windows

```
:: With Visual Studio (Developer Command Prompt):
cl /O2 /DUNICODE /D_UNICODE DisplayFix.c user32.lib gdi32.lib /Fe:DisplayFix.exe
:: With MinGW-w64 for Windows:
gcc -O2 -municode -o DisplayFix.exe DisplayFix.c -lgdi32 -luser32
```

> The repository ships the source; a pre-built binary may or may not be present depending on whether it was built (see `scripts/windows/README.md`). If absent, build it with the two commands above, or just use the PowerShell scripts, which need no compiler.

6.7 Headless / remote servers

If the ML10 runs as a headless server, the physical console mode is mostly irrelevant — use **RDP**, which renders at the client's resolution regardless of the local adapter:

- Enable: `Server Manager → Local Server → Remote Desktop`, or `powershell Set-ItemProperty 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -Name fDenyTSConnections -Value 0 Enable-NetFirewallRule -DisplayGroup "Remote Desktop"`
- For a server with **no desktop experience** and no GPU driver, RDP still gives you a usable desktop at your chosen size.

6.8 Windows 10 / 11 (workstation)

Simpler than Server:

1. Let **Windows Update** install a graphics driver, or run the Intel **Driver & Support Assistant**.
2. If the monitor is not detected through a passive adapter, install an **active DP adapter**.
3. Resolution: *Settings → System → Display*, or `DisplayFix.exe --best`.

Windows 10/11 have full WDDM support and hardware acceleration for the P530 with the 15.45 or newer driver.

6.9 Windows checklist

- [ ] Confirm `Ven_8086&Dev_191D` is present (Device Manager → Details → Hardware Ids).
- [ ] Install a real Intel driver (no more "Microsoft Basic Display Adapter").
- [ ] Confirm EDID is read (monitor shows a name in Display settings, not "Generic PnP Monitor").
- [ ] If not, replace the passive adapter with an **active** one.

- [ ] Pin the resolution (Display settings / script / `DisplayFix.exe`).
- [ ] For unattended servers, prefer RDP and ignore the physical console.

7. Other operating systems

The root causes (EDID over a passive adapter; no `nomodeset` concept elsewhere) recur on every OS. Below is what to do on the platforms people actually run on an ML10 Gen9-class box.

7.1 FreeBSD

FreeBSD uses the **`i915kms`** kernel module (a port of the Linux driver). There is no `nomodeset`.

```
# /boot/loader.conf
i915kms_load="YES"
kern.vty=vt                 # use the new vt(4) console
```

Load immediately without rebooting:

```
kldload i915kms
```

Check:

```
pciconf -lv | grep -A4 vgapci
sysctl kern.vty
dmesg | grep -i i915
```

If the monitor is not detected through the adapter, force the mode:

```
# after X starts, or from a script:
xrandr                 # list outputs (often HDMI-1 / DP-1)
xrandr --output HDMI-1 --mode 1366x768
```

FreeBSD has no `video=` kernel token; use an **active adapter** for reliable detection, or an `/etc/X11/xorg.conf.d/` `PreferredMode` stanza as in Linux.

A helper is provided: `scripts/freebsd/i915-freebsd.sh`.

7.2 Proxmox VE / Debian hypervisor

Proxmox is Debian under the hood → the **Linux fix applies directly**:

```
sudo ./scripts/linux/i915-display-fix.sh apply --connector HDMI-A-1 --mode 1366x768
sudo reboot
```

If the host is headless, the local console barely matters; the fix is mainly so the boot console and any local login are usable.

7.3 VMware ESXi

ESXi is a hypervisor, not a desktop. The integrated GPU is not used; ESXi runs a VGA/EFI framebuffer at low resolution for its DCUI (“yellow console”). This is by design — you manage ESXi over the network (Host Client / vSphere).

- If the DCUI is unreadable through an adapter, add an **active adapter** or use IPMI/remote management.
- Passing the iGPU to a VM (GTV-d / Intel GVT-g) is not supported on Skylake P530 in ESXi’s standard tooling; use a discrete GPU for that.

7.4 TrueNAS SCALE / CORE, Unraid, OMV

All are Linux:

- **TrueNAS SCALE / Unraid / OMV** (Debian/RHEL-derived): run the Linux toolkit. On TrueNAS SCALE the root filesystem is managed, but kernel parameters can be added via the UI (System → Advanced → **Kernel command line / sysctl**) or `/etc/default/grub` + `update-grub` from a shell.
- **TrueNAS CORE** (FreeBSD): follow §7.1.

For appliance-style NAS boxes with no keyboard/video session, the practical fix is again the **active adapter**.

7.5 pfSense / OPNsense (FreeBSD)

Same as FreeBSD (§7.1): load `i915kms`, and force the mode if needed. These router images are usually headless; the console is only used once.

7.6 illumos / OpenIndiana / SmartOS

Intel KMS support is limited/experimental. On Skylake P530 expect the generic VGA framebuffer. Use an active adapter, and drive the machine over the network. Not a supported display target.

7.7 ChromeOS Flex / other Linux appliances

ChromeOS Flex uses the standard Intel graphics stack. Boot parameters are not user-editable in the usual way; rely on the **active adapter**.

7.8 Dual-boot / multi-OS on the same machine

Because the fix is per-OS, you can run Linux (fixed via kernel line) and Windows/Server (fixed via driver + adapter) side by side on the ML10 Gen9. The **active adapter** benefits every OS at once and is the recommended baseline.

7.9 Summary table

OS	Mechanism	Best fix
Linux (any distro)	`i915` + `video=` on kernel cmdline	This repo’s toolkit
Windows / Server	Intel WDDM driver	Driver + **active adapter**
FreeBSD / pfSense / TrueNAS CORE	`i915kms`	`i915kms_load=YES` (+ active adapter)
Proxmox / TrueNAS SCALE / Unraid	Linux	Linux toolkit
ESXi	hypervisor framebuffer only	Manage over network; active adapter for DCUI
illumos	limited KMS	active adapter, manage over network

8. Hardware, cables, adapters & alternative solutions

Sometimes the cleanest fix is physical. This document explains the options and why they work.

8.1 What the ML10 Gen9 actually has

The motherboard has **2 × DisplayPort 1.2** and **no HDMI/DVI/VGA**. Per the GPU VBT (see examples/i915-vbt-ports.txt), ports B/C/D are **DP++** (dual-mode), i.e. they can emit native DisplayPort **or** TMDS (DVI/HDMI) signals.

Two implications:

1. A **passive** DP→HDMI/DVI adapter is *electrically allowed* (DP++), which is why the machine sometimes gives a picture at all.
2. But passive adapters have no chip to regenerate **DDC/EDID**, so detection is unreliable → the failure in this repo.

> HP's own documentation for the ML10 Gen9 states the DP ports require an **active** dongle. This is the "official" hardware answer.

8.2 Passive vs active adapters

	Passive DP→HDMI/DVI	**Active DP→HDMI/DVI**
Chip inside	none	level shifter / protocol converter
Works when source is DP++	sometimes	always
Passes EDID/DDC reliably	often **no**	yes
Fixes this bug	no	**yes**
Linux `video=` workaround needed	yes	usually not

How to tell them apart: active adapters are usually slightly larger, contain a chip, are labelled "active", "DP++ to HDMI", or explicitly "for NVIDIA/AMD/ non-DP++". A "DP to HDMI passive" plug is the one causing trouble.

8.3 Recommended cables/adapters

In rough order of preference:

1. **Active DP → HDMI cable/adapter**, then a normal HDMI cable to the monitor. One hop, EDID regenerated, works on the ML10 out of the box.
2. **Active DP → DVI**, then DVI→HDMI cable. Also fine; two hops but the active DP→DVI regenerates DDC.
3. **Active DP → VGA** (if you must drive a VGA monitor; degrades quality).
4. **USB graphics adapter** (DisplayLink) — an entirely different path that bypasses the iGPU output; requires drivers on every OS.
5. **A monitor with DisplayPort** — remove the adapters entirely; the cleanest of all if you are buying hardware.

Avoid: passive DP→HDMI/DVI plugs, “DP to HDMI” cables with no chip, unpowered KVMs with DP inputs that do not support DP++.

8.4 KVM switches

Many KVM switches present a **fake EDID** to the computer so switching does not disturb the display. That is actually helpful here (it removes the EDID race). Caveats:

- A KVM that does **not** emulate EDID behaves like a bad adapter.
- Prefer KVMs advertised as “supports DP++ / EDID emulation”.
- Some KVMs force a low maximum mode (e.g. 1920×1080); verify it passes your native resolution.

8.5 A discrete GPU alternative

If you need more outputs or guaranteed HDMI:

- Any cheap low-profile card (e.g. a GT 1030 / RX 6400 / Intel Arc A310) gives native HDMI/DP and sidesteps the iGPU entirely.
- The ML10 Gen9 has one PCIe x16 slot (electrically x8/x16 depending on config) and a modest PSU; check power and clearance first.
- On Windows Server this also often gets you a driver that actually installs, because discrete vendors ship server-adjacent drivers.

8.6 BIOS/firmware settings worth checking

In the ML10 Gen9 BIOS (F10 at boot):

- **Embedded Video / Onboard VGA** — ensure it is enabled (there is no discrete GPU, so this should default on).
- **Primary display** — if a discrete card is added, set which one POSTs.
- **Legacy vs UEFI video** — UEFI generally gives a better GOP framebuffer; the reference machine boots UEFI.
- **BIOS version** — the reference machine is on **1.010**; newer BIOS revisions occasionally improve iGPU POST behaviour.

A BIOS update is not a fix for this bug, but it is worth having current firmware.

8.7 What *not* to do

- **Do not rely on `nomodeset`** as a “fix”. It permanently caps you at the firmware resolution with no acceleration.
- **Do not disable the iGPU** to “avoid conflicts” — you have no other GPU.
- **Do not buy a second passive adapter** hoping a different brand behaves better; the mechanism is the same. Buy *active*.

8.8 Decision guide

```
flowchart TD
  A[No/ugly picture on ML10 Gen9] --> B{Can you replace the adapter?}
  B -- Yes --> C[Buy an ACTIVE DP->HDMI adapter]
  C --> D[Works on Linux, Windows, FreeBSD]
  B -- No / not yet --> E{Linux?}
  E -- Yes --> F[Use this repo's video= fix]
  E -- No --> G[Windows/other: install driver + registry/script force mode]
  D --> H[Done]
  F --> H
  G --> H
```

9. Troubleshooting

Work top to bottom. Almost every “still dark” case is one of: `nomodeset` still present, wrong connector name, wrong mode, or a passive adapter.

9.1 Quick checks

```
cat /proc/cmdline                    # is nomodeset gone? is video=<name>:<mode>e there?
grep -qw i915 /proc/modules          # is i915 loaded?
for c in /sys/class/drm/*/status; do echo "$(basename "$(dirname "$c")"): $(cat "$c")"; done
xrandr | grep connected
journalctl -k -b | grep -iE 'i915.*(fail|error|WARN)' | grep -vi AUX
```

9.2 Symptom → cause → fix

Symptom	Likely cause	Fix
`modprobe i915` → “No such device”; only `simpledrm`	`nomodeset` still on the cmdline	Remove it, reboot (see 04)
“No signal” / monitor sleeps, nothing at all	Connector not force-enabled, EDID missed	Add `video=<correct-name>:<mode>e`
Signal present, **black screen, cursor flickers**	You forced **both** `DP-x` and `HDMI-A-x` (same port)	Force **only** the one matching the cable
Picture but 1024×768 / 640×480	Mode not forced; firmware framebuffer in use	Force the native mode; check `xrandr`
Correct on one boot, wrong on the next	EDID/bit-bang race at boot	Keep the `e` suffix; consider an active adapter
Works at login screen, goes black after login	Desktop picked a bad mode/scale	Pin the mode (`xrandr` /Xorg/autostart)
`i915` loads, no connector `connected`	EDID not read at all (passive adapter)	Use `discover`, then an **active** adapter
Wrong resolution only in the installer/live USB	Live images may default to `nomodeset`	Add the same `video=` token to the boot menu (`e` to edit, or `Tab`)
Flicker / random blanks at idle	Skylake power features	Keep `i915.enable_dc=0 enable_psr=0 enable_fbc=0`
Second monitor not detected	Only one DDI active, or MST misconfig	Enable the right second connector; keep `enable_dp_mst=0` for a hub-free setup

---

9.3 (a) The screen is dark and I don’t know the connector name

This is the “chicken and egg” case: you cannot read `/sys/class/drm` names until `i915` is running, but you cannot see anything when it is.

Option 1 — SSH (best)

Before rebooting, enable SSH so you can log in while the screen is dark:

```
sudo systemctl enable --now ssh
# note the IP:
ip -brief addr
```

Then boot without `nomodeset` (or use the toolkit’s `discover` entry) and, from another machine:

```
ssh you@<ip> 'for c in /sys/class/drm/*/status; do echo "$(basename "$(dirname "$c")"): $(cat "$c")"; done
```

Whatever is `connected` is your connector name.

Option 2 — the toolkit’s discover entry

```
sudo ./i915-display-fix.sh discover            # adds a non-destructive GRUB entry
# reboot, pick "i915 DISCOVER - no nomodeset"
./i915-display-fix.sh detect                   # prints the connector + apply command
```

If even that stays dark, try:

```
sudo ./i915-display-fix.sh discover --force-all
```

`--force-all` force-enables every candidate port. It may still be black (multi connector conflict), but it lets you SSH in and read which connector reports `connected`/has an EDID. Then apply with that single connector.

Option 3 — guess from the port

If your cable is in the **left** DP port, try `HDMI-A-1` first; the **right** port, try `HDMI-A-2`. You can pass both as separate attempts, rebooting between.

9.4 (b) “Signal but black screen, cursor flashes”

You are forcing two names for one port. Remove the `video=` tokens and keep only the one matching the cable:

```
-video=DP-1:1366x768@60e video=HDMI-A-1:1366x768@60e
+video=HDMI-A-1:1366x768@60e
```

Confirm in the log:

```
journalctl -b -1 -k | grep -i "handle_conflicting_encoders"
```

9.5 (c) Wrong resolution

Find the native mode:

```
xrandr                      # look at the '*' (current) and the list for the connected output
# or, without a desktop:
cat /sys/class/drm/card*-HDMI-A-1/modes | head
```

Set it live, then make it permanent:

```
xrandr --output HDMI-1 --mode 1366x768          # live test
# permanent: change the number in video=HDMI-A-1:1366x768@60e and re-run update-grub
```

9.6 (d) Recovery when the screen stays black

1. **Power-cycle.** The GRUB menu (if it appears) lets you pick *“Recovery - safe graphics (nomodeset)”* or the distro’s *recovery mode*.
2. If you cannot see the menu at all, force the recovery entry: - At the GRUB menu, press `e`, append `nomodeset` to the `linux` line, `Ctrl-X` / `F10` to boot.
3. Once in, revert:

```
sudo ./i915-display-fix.sh revert
sudo reboot
```

4. Last resort: boot a live USB and edit `/etc/default/grub` on the installed system (mount the root partition, edit, chroot or just edit the file, then regenerate GRUB).

9.7 Diagnostic cheat-sheet

```
# GPU + driver
lspci -nnk | grep -A3 -iE 'vga|display'
# Kernel + cmdline
uname -r; cat /proc/cmdline
# Module state
grep -w i915 /proc/modules || echo "i915 not loaded"
# Connectors
for c in /sys/class/drm/*/status; do echo "$(basename "$(dirname "$c")"): $(cat "$c")"; done
# Modes per connector
cat /sys/class/drm/card*-HDMI-A-1/modes
# EDID presence
ls -l /sys/class/drm/card*-HDMI-A-1/edid
# Full DRM debug (verbose; needs drm.debug=0x1e on the cmdline)
journalctl -b -k | grep -iE 'i915|drm' | less
```

The automated collector is `scripts/linux/i915-collect.sh`.

9.8 Harmless log noise (do not chase)

```
intel_hdmi_set_edid: HDMI GMBUS EDID read failed, retry using GPIO bit-banging
failed to retrieve link info, disabling eDP
PSR condition failed: Port not supported
Reserved GGTT:[...] for use by error capture
```

These mean: EDID came in over bit-banging, there is no internal panel, and PSR isn’t applicable. They are expected on this hardware.

10. Verification

How to prove the fix worked, per platform. Run these after rebooting into the fixed configuration.

10.1 Linux — one-liners

```
# 1. Kernel: nomodeset is gone, the video= token is present
cat /proc/cmdline

# 2. The real driver is loaded
grep -qw i915 /proc/modules && echo "i915 OK" || echo "i915 MISSING"

# 3. The connector is connected and enabled at the right mode
for c in /sys/class/drm/*/status; do
  d=$(dirname "$c")
  echo "$(basename "$d"): $(cat "$c") enabled=$(cat "$d/enabled") mode=$(head -1 "$d/modes")"
done

# 4. Xorg sees it at the right resolution
xrandr | grep connected

# 5. Hardware acceleration (not llvmpipe)
glxinfo | grep -E 'OpenGL renderer|direct rendering'
```

Expected (reference machine):

```
HDMI-A-1: connected enabled=enabled mode=1366x768
HDMI-1 connected primary 1366x768+0+0
OpenGL renderer string: Mesa Intel(R) HD Graphics P530 (SKL GT2)
direct rendering: Yes
```

Or simply:

```
./scripts/linux/i915-display-fix.sh status
```

Automated pass/fail check

```
# fails loudly if nomodeset is still present or i915 is missing
! grep -qw nomodeset /proc/cmdline && grep -qw i915 /proc/modules \
  && echo "PASS" || echo "FAIL"
```

10.2 Finding your native resolution

```
# Preferred (first) mode the monitor advertises:
cat /sys/class/drm/card*-HDMI-A-1/modes | head -1

# Everything it supports:
cat /sys/class/drm/card*-HDMI-A-1/modes

# The raw EDID (decode with edid-decode if installed):
ls -l /sys/class/drm/card*-HDMI-A-1/edid
edid-decode /sys/class/drm/card*-HDMI-A-1/edid | grep -i preferred -A2
```

The first mode in `modes` is the monitor's preferred/native mode — use it in the `video=` token.

10.3 Windows / Server

```
Get-CimInstance Win32_VideoController |
  Select-Object Name, DriverVersion, CurrentHorizontalResolution,
                CurrentVerticalResolution, CurrentRefreshRate

# confirm it is not the basic adapter:
Get-PnpDevice -Class Display | Format-Table Status, FriendlyName, InstanceId
```

Good: `Intel(R) HD Graphics ...` at your native resolution, not `Microsoft Basic Display Adapter`.

The provided tools:

```
powershell -ExecutionPolicy Bypass -File scripts\windows\Get-GraphicsInfo.ps1
DisplayFix.exe --current
```

10.4 FreeBSD

```
sysctl kern.vty                      # should say vt
kldstat | grep i915kms
pciconf -lv | grep -A4 vgapci
# under X:
xrandr | grep connected
```

10.5 Performance sanity check

A properly driven display has hardware acceleration. If `glxinfo` says `llvmpipe` (Linux) or Windows shows the Basic Display Adapter, the fix is *not* complete even if the resolution looks right.

```
# Linux: should NOT say llvmpipe / softpipe
glxinfo | grep -i 'renderer'

# Quick OpenGL capability
glxinfo | grep 'OpenGL version'
```

10.6 What “success” looks like

Check	Expected
`/proc/cmdline`	no `nomodeset`; has `video=<connector>:<WxH>@<Hz>e`
Driver	`i915` loaded (Linux) / Intel driver (Windows)
Connector	exactly one `connected`, `enabled`
Current mode	monitor’s native resolution
Renderer	Intel GPU, hardware accelerated
Stability	no flicker / blanking over several minutes idle

11. Reference

11.1 Kernel command line parameters used

Parameter	Values	Meaning
`nomodeset`	*(presence)*	Blocks DRM/KMS drivers (sets `video_firmware_drivers_only()`). **Remove it.**
`video=<conn>:<WxH>[@Hz][e]`	see §11.2	Force-enable an output and its mode.
`i915.enable_dc=`	`0` / `1` / `2`	Display C-states (DC5/DC6). `0` = off (stable).
`i915.enable_psr=`	`0` / `1`	Panel Self Refresh. `0` = off.
`i915.enable_fbc=`	`0` / `1`	Frame Buffer Compression. `0` = off.
`i915.enable_dp_mst=`	`0` / `1`	DP Multi-Stream Transport. `0` = off.
`i915.disable_power_well=`	`0` / `1` / `-1`	`0` = keep power wells always on.
`drm.debug=`	hex bitmask	DRM debug categories (see §11.4).
`log_buf_len=`	e.g. `16M`	Enlarge the kernel log ring (for verbose debug).
`quiet`	—	Suppress most boot messages.
`splash`	—	Graphical boot splash.

11.2 `video=` syntax in full

```
video=<connector>:<xres>x<yres>[@<refresh>][<options>]
```

- `<connector>` — DRM connector name: `DP-1`, `HDMI-A-1`, `eDP-1`, `VGA-1`, … Use the name from `/sys/class/drm/`.
- `<refresh>` — optional vertical refresh in Hz, e.g. `@60`.
- `<options>` — one or more of:

Suffix	Meaning
`e`	Force-enable the output even if the driver thinks it is disconnected. **This is the key suffix for this bug.**
`d`	Force-**disable** the output.
`D`	Enable with **digital** (TMDS) signalling — useful to force an HDMI/DVI encoder.
`m`	Use mode with margins.
`i`	Interlaced.

Examples:

```
video=HDMI-A-1:1366x768@60e      # force 1366x768@60 on HDMI-A-1, even if "disconnected"
video=DP-1:d                     # force-disable DP-1
video=HDMI-A-1:1920x1080@60eD    # force digital + 1080p
```

You can list several `video=` tokens, but **do not** force both names of the same physical port (see 03).

11.3 Connector name ↔ X output name

DRM (kernel)	Xorg (`xrandr`)	Type
`HDMI-A-1`	`HDMI-1`	HDMI/DP++ in TMDS mode
`DP-1`	`DP-1`	native DisplayPort
`eDP-1`	`eDP-1`	internal panel
`VGA-1`	`VGA-1`	analog

`HDMI-A-2`, `DP-2`, … correspond to the next physical port.

11.4 `drm.debug` bitmask

Bit	Category
`0x01`	`DRM_UT_CORE`
`0x02`	`DRM_UT_DRIVER`
`0x04`	`DRM_UT_KMS` (connector/EDID/modeset — the useful one)
`0x08`	`DRM_UT_PRIME`
`0x10`	`DRM_UT_ATOMIC`
`0x1e`	DRIVER+KMS+PRIME+ATOMIC (what this repo used)

Debug messages are rate-limited and verbose; turn them off (remove `drm.debug`) once you are done.

11.5 Useful commands

```
# DRM state
cat /sys/class/drm/card*-*/status
cat /sys/class/drm/card*-*/modes
cat /sys/class/drm/card*-*/enabled

# Xorg
xrandr --verbose
xrandr --output HDMI-1 --mode 1366x768 --rate 60
cvt 1366 768 60              # generate a modeline
gtf 1366 768 60

# Kernel / module
modinfo i915
for p in /sys/module/i915/parameters/*; do echo "$(basename $p)=$(cat $p)"; done

# Boot
bootctl status               # systemd-boot
grub-mkconfig -o /boot/grub/grub.cfg
```

11.6 Glossary

Term	Meaning
KMS	Kernel Mode Setting — the kernel programs the display controller.
i915	Linux kernel driver for Intel integrated graphics.
nomodeset	Kernel flag that disables KMS; leaves the firmware framebuffer.
EDID	Monitor capability block read over DDC/I²C.
DDC / DDC-CI	Display Data Channel (the I²C bus carrying EDID).
GMBUS	Intel's hardware I²C controller for DDC.
Bit-banging	Software-toggling the DDC lines when GMBUS fails (slow).
DP++	Dual-mode DisplayPort: can output native DP or TMDS (HDMI/DVI).
DDI	Digital Display Interface — the physical Intel display port block.
DDC	see above
CRTC	Display controller / scanout pipe.
PHY	The electrical layer of a display port.
PSR	Panel Self Refresh (power saving).
FBC	Frame Buffer Compression.
MST	Multi-Stream Transport (daisy-chaining DP displays).
DCH	Declarative Componentized Hardware — modern Windows driver model.
WDDM	Windows Display Driver Model.

11.7 File locations (Linux)

Path	Purpose
`/etc/default/grub`	GRUB defaults (edit here)
`/boot/grub/grub.cfg`	generated GRUB (Debian/Arch)
`/boot/grub2/grub.cfg`	generated GRUB (Fedora/openSUSE)
`/etc/grub.d/`	scripts that generate menu entries (never put backups here)
`/boot/loader/entries/*.conf`	systemd-boot entries
`/sys/class/drm/`	live connector state
`/var/backups/i915-display-fix/`	backups made by this repo’s toolkit
`/etc/X11/xorg.conf.d/`	Xorg snippets (mode pinning)

12. FAQ

Q: Is my ML10 Gen9's iGPU actually supported? Yes. The HD Graphics P530 (`8086:191d`) is fully supported by Linux `i915` and by the Windows Intel graphics driver. HP does not ship a Windows Server driver for it, which is why it "looks" unsupported.

Q: Why does `nomodeset` "work" then? Because it does not use the GPU at all — it uses the BIOS's simple framebuffer (`simpledrm`). That is why you are stuck at 1024×768 with no acceleration.

Q: Can I fix this without rebooting? No. `nomodeset` is read once at boot. Live `modprobe i915` fails while it is set.

Q: Do I really need the `i915.enable_*` flags? Usually only `video=<conn>:<WxH>@<Hz>e` (and removing `nomodeset`) is required. The flags are conservative stability tweaks for Skylake; keep them if you saw flicker, drop them if you prefer default power behaviour and it still works.

Q: Why do I have to name `HDMI-A-1` when the port is a DisplayPort? Because a DP++ port is dual-mode. In HDMI/DVI mode the kernel exposes it under the `HDMI-A-x` name. `DP-1` and `HDMI-A-1` are the same physical port.

Q: I forced both and got a black screen with a cursor. Why? Two connectors on one port fight over the shared encoder (`handle_conflicting_encoders`), so the driver keeps disabling your monitor. Force only one.

Q: Can I use my monitor's full 1920×1080? Yes, if the monitor and adapter support it. Put the native mode in the token: `video=HDMI-A-1:1920x1080@60e`. Verify the mode exists: `cat /sys/class/drm/card*-HDMI-A-1/modes`.

Q: Will this survive kernel updates? Yes. `/etc/default/grub` is static, and `update-grub` / `grub2-mkconfig` regenerates from it for every installed kernel. The optional custom recovery entry hardcodes the kernel of the day; re-run `apply` after a major upgrade, or rely on your distro's built-in recovery entry.

Q: What about Secure Boot? The fix does not touch boot signing; it only changes the kernel command line. It works with Secure Boot enabled. The Windows `.exe` approach is unaffected.

Q: Does this help on Windows? Not directly. On Windows install the Intel driver and use an **active** adapter; the `video=` concept does not exist there. See docs/06-windows-server.md.

Q: Is a passive adapter really the problem? It is the *root* of the EDID failure. Passive DP→HDMI/DVI plugs do not regenerate DDC. An active adapter removes the need for the software workaround.

Q: I only see 640×480. Is that a different bug? No. It is the same: without EDID the driver/OS falls back to a safe low mode. Force the mode, then consider an active adapter.

Q: Can I disable `nomodeset` but keep text-only boot? Yes — use `nomodeset` only for the recovery entry; the normal entry should not have it. That is exactly what this repo sets up.

Q: How do I go back? `sudo ./scripts/linux/i915-display-fix.sh revert`, or restore the `.bak` of `/etc/default/grub` and `update-grub`. A recovery GRUB entry is always left in place.

Q: Why did my GRUB menu get a duplicate/garbled entry after editing? Because a `*.bak` file was left inside `/etc/grub.d/`, which GRUB executes. Keep backups elsewhere (the toolkit uses `/var/backups/i915-display-fix/`).

Q: Will this work on a monitor with DisplayPort directly (no adapter)? Yes, and it is the best case. You may not even need the `video=` force; plain `nomodeset`-removal is likely enough.

Q: Is there a risk of bricking the display permanently? No. All changes are file-based and reversible, and a `nomodeset` recovery entry is always available. Even in the worst case you reach a console with a live USB.

Changelog

All notable changes to this project are documented here. Format loosely follows Keep a Changelog.

[Unreleased]

Added

- **GitHub community-health files**: `dependabot.yml` (weekly `github-actions` updates), issue forms `bug_report.yml` and `success_report.yml`, an `ISSUE_TEMPLATE/config.yml` linking the key docs, and a pull-request template.
- **Social images**: `assets/social/social-1200x627.png` (landscape / link preview) and `social-1080x1080.png` (square feed post), regenerated by `scripts/doc/make-social.sh`.
- `Makefile` `social` target; `docs` now also builds the social images.

Changed

- **Windows driver installation is now a first-class, scriptable step** like any other INF/`pnputil` driver install, on Windows 10/11 and Server 2012 R2–2025: `Install-IntelGraphicsDriver.ps1` now self-elevates, auto-detects the extracted Intel package when `-Path` is omitted, and validates the INF. Added a double-click wrapper `Install-IntelGraphicsDriver.bat`, plus `install` and `drivers` subcommands in `Fix-Display.bat`; documented in `docs/06-windows-server.md` and `scripts/windows/README.md`.

[1.1.0] - 2026-10-14

Added

- **`QUICKSTART.md`** — the 60-second version of the whole fix.
- **Citation metadata**: `CITATION.cff` (enables GitHub's "Cite this repository") and `AUTHORS` with a ready-made BibTeX entry.
- **Printable PDF** of the complete documentation: `docs/dist/ProLiant-ML10-Graphics-fix.pdf` (52 pages, A4), built from the Markdown sources with a cover page and table of contents.
- **Screenshots section** in the README with real terminal captures under `assets/screenshots/` (toolkit status, connector detection, `xrandr`, `glxinfo`, proof panel, Windows tools).
- **Social-preview / README banner**: `assets/social-preview.png` (1280×640), regenerated by `scripts/doc/make-banner.sh`.
- **Documentation build scripts** `scripts/doc/build-pdf.sh`, `scripts/doc/make-screenshots.sh` and `scripts/doc/make-banner.sh` (Markdown → HTML → PDF via headless Chrome; real command output rendered to PNG; banner render).
- `Makefile` targets `pdf`, `screenshots`, `banner`, `docs`.

Changed

- README: added a hero banner, quick-start/PDF callouts, a Screenshots section, a Citation section, and an updated repository-layout diagram.
- Makefile `lint` now also covers `scripts/doc/*.sh`.
- CI lint covers the doc scripts and builds the PDF as an artifact.

[1.0.0] - 2026-10-14

Initial release.

Added

- Complete root-cause analysis for the ML10 Gen9 (Intel HD Graphics P530) display failure over a DP→DVI→HDMI chain.
- `nomodeset` behaviour and `video_firmware_drivers_only()`.
- EDID/GMBUS failure and the GPIO bit-bang fallback.
- The dual-name DP++ port trap (`DP-1` vs `HDMI-A-1` on the same DDI).
- Skylake power-feature stability flags.
- **Linux toolkit** `scripts/linux/i915-display-fix.sh` with `status`, `detect`, `apply`, `discover`, `revert`; supports GRUB (Debian/Ubuntu/Mint, Fedora/RHEL, Arch, openSUSE) and systemd-boot.
- `scripts/linux/i915-collect.sh` diagnostic collector.
- `scripts/linux/finalize-i915.sh` one-shot applier.
- **Windows / Windows Server** tooling:
- `Get-GraphicsInfo.ps1`, `Set-DisplayResolution.ps1`, `Install-IntelGraphicsDriver.ps1`, `Fix-Display.bat`.
- `DisplayFix.exe` (native, ~44 KB, built from `src/DisplayFix.c`) plus source.
- **FreeBSD** helper `scripts/freebsd/i915-freebsd.sh`.
- Full documentation set (`docs/01`–`docs/12`) covering problem, diagnosis, root cause, Linux solution, per-distro guide, Windows Server, other OSes, hardware/cables/adapters, troubleshooting, verification, reference and FAQ.
- Real diagnostic captures in `examples/`.
- `Makefile` (build the Windows `.exe`, lint), `install.sh`, CI workflow.

Verified on

- HPE ProLiant ML10 Gen9, BIOS 1.010, Xeon E3-1225 v5, HD Graphics P530.
- Linux Mint 22.3, kernels 7.0.0-38-generic and 6.14.0-37-generic.
- Monitor 1366×768 via DP→DVI→HDMI. Result: 1366×768 @ 60 Hz, i915 + Mesa.